



Recursion

[Visit our website](#)

Introduction

WELCOME TO THE RECURSION TASK!

Recursion is a handy programming tool that, in many cases, enables you to develop a straightforward, simple solution to an otherwise complex problem. However, it is often difficult to determine how a program can be approached recursively. In this task, we explain the basic concepts of recursive programming and teach you to “think recursively”.

WHAT IS RECURSION?

When faced with a particularly difficult or complex problem, it is often easier to break the problem down into smaller, more manageable chunks that are easier to solve. This is the basic idea behind recursion. Recursive algorithms break down a problem into smaller pieces that you already know how to solve.

In simple terms, recursion is when a function calls itself. Normally a recursive function uses conditional statements to call the function recursively or not. The main benefits of using recursion are **compact** code, **easily understandable** code, and **fewer variables** are used. Recursion and iteration (loops) can be used to achieve the same results. However, unlike loops, which work by explicitly specifying a repetition structure, recursion uses continuous function calls to achieve repetition.

Recursion is a somewhat advanced topic, and problems that can be solved with recursion can also most likely be solved by using simpler looping structures. However, recursion is a useful programming technique that, in some cases, can enable you to develop natural, straightforward, simple solutions to otherwise difficult problems.

The following guidelines will help you to decide which method to use depending on a given situation:

- **When to use recursion?** When a compact, understandable and intuitive code is required.
- **When to use iteration?** When there is limited memory and faster processing is required.

RECURSIVE FUNCTIONS

As mentioned previously, a recursive function is a function that calls itself. For example, let's say that you have a cake that you wish to share equally amongst several friends. To do so, you might start by cutting the cake in half and then again cutting the resulting

slices in half until there are enough slices for everyone. The code to implement such an algorithm might look something like this:

```
public static int cutCake(int numberOfFriends, int numberOfSlices) {
    // Cut cake in half
    numberOfSlices = numberOfSlices * 2;

    // Check if there are enough slices for everybody
    if (numberOfSlices >= numberOfFriends) {
        // If there are enough slices - return the number of slices
        return numberOfSlices;
    }
    else {
        // If there are not enough slices - cut the resulting
        // slices in half again.
        return cutCake(numberOfFriends, numberOfSlices);
    }
}

public static void main(String[] args) {
    System.out.println(cutCake(11, 1));
}
```

The `cutCake` function takes the number of friends you wish to share the cake with (11) and the number of slices of cake (initially 1, since the cake is not cut). Line 4, `numberOfSlices = numberOfSlices * 2`, cuts the cake in half. Line 7 then checks if there are enough slices. If there are enough slices, the number of slices is returned (line 9) and if not the function calls itself again (line 14) to cut the cake in half one more time. This is an example of a recursive function.

COMPUTING FACTORIALS USE RECURSION

Many mathematical functions can be defined using recursion. A simple example is the **factorial function**. The factorial function ($n!$) describes the operation of multiplying a number by all positive integers less than or equal to itself. For example, $4! = 4 \times 3 \times 2 \times 1$

If you look closely at the examples above you might notice that $4!$ can also be written as $4! = 4 \times 3!$. In turn, $3!$ can be written as $3! = 3 \times 2!$ and so on. Therefore, the factorial of a number n can be recursively defined as follows:

$$0! = 1$$

$$n! = n \times (n - 1)! \text{ where } n > 0$$

Assuming that you know $(n - 1)!$, you can easily obtain $n!$ by using $n \times (n - 1)!$. The problem of computing $n!$ is, therefore, reduced to computing $(n - 1)!$. When computing $(n - 1)!$ you can apply the same idea recursively until n is reduced to 0. The recursive function for calculating $n!$ is shown below:

```
public static long factorial(int n){
    // Base case: 0! = 1
    if (n == 0) {
        return 1;
    }
    else {
        // Recursive case: n! = n * (n-1)!
        return n * factorial(n-1);
    }
}
```

If you call the function **factorial(n)** with $n = 0$, it immediately returns a result of 1. This is known as the **base case** or the stopping condition. The base case of a function is the problem to which we already know the answer. In other words, it can be solved without any more recursive calls. The base case is what stops the recursion from continuing forever. Every recursive function must have at least one base case.

If you call the function **factorial(n)** with $n > 0$, the function reduces the problem into a subproblem for computing the factorial of $n - 1$. The subproblem is essentially a simpler or smaller version of the original version. Because the subproblem is the same as the original problem, you can call the function again, this time with $n - 1$ as the argument. This is referred to as a recursive call. A **recursive call** can result in many more recursive calls because the function keeps on dividing a subproblem into new subproblems. For a recursive function to terminate, the problem must eventually be reduced to a base case.

In summary, there are two main requirements for a recursive function:

- **Base case:** the function returns a value when a certain condition is satisfied, without any other recursive calls.
- **Recursive call:** the function calls itself with an input which is a step closer to the base case.

Practical Task 1

Follow these steps:

- In a file called **palindromeRecursion.java**, create:
 - o a recursive function that determines whether a given array is a palindrome.
 - o Compile, save and run your file.

Be sure to place files for submission inside your **task folder** and click "Request review" on your dashboard.

Practical Task 2

Follow these steps:

- In a file called **LCMRecursion.java**, create:
 - o a recursive function to calculate the lowest common multiple (LCM) for two numbers x and y using recursion.
 - o LCM is the smallest positive integer that is a multiple of both the numbers.
 - o Compile, save and run your file.

Be sure to place files for submission inside your **task folder** and click "Request review" on your dashboard.

Optional Bonus Task

Follow these steps:

- Create a file called **searchReplace.java**. In here, create a program that implements a search and replace function recursively. Your program should allow a user to enter a string, a substring they wish to find and another string with which they wish to replace the found substring.

The output of your program should be similar to the output given below:

Please enter a string: Hello world

Please enter the substring you wish to find: llo

Please enter a string to replace the given substring: @@
Your new string is: he@@ world

Be sure to place files for submission inside your task folder and click "Request review" on your dashboard.



Share your thoughts

HyperionDev strives to provide internationally excellent course content that helps you achieve your learning outcomes.

Do you think we've done a good job or do you think the content of this task, or this course as a whole, can be improved?

Share your thoughts anonymously using this [form](#).
